National University of Computer and Emerging Sciences

CS3009 — Software Engineering | Spring 2026

Naawbi

Learning Management System

Deliverable 3

Sprint 3 (Final Sprint) | Scrum Report + Deliverable Document

Team Name	**Ibwaan**
Team Lead	**Ibraheem Farooq**
Members	23i-0708 | Ali Aan Khowaja 23i-0816 | Ibraheem Farooq 23i-0819 | Wajih-Ur-Raza Asif
Submission Date	20 April 2026
Repository	github.com/aliaankhowaja/Naawbi

Contents

1 Sprint 3 Overview **2**

2 Sprint 3 Backlog **2**
2.1 Handoff from Sprint 2 . . . 2
2.2 Module Selected for Sprint 3 . . . 2
2.3 User Stories Selected for Sprint 3 . . . 2
2.4 Sub User Stories and Task Breakdowns . . . 3
2.5 Structured Specifications for Four User Stories . . . 4

3 Scrum Board (GitHub Projects) **5**

4 Sprint 3 Burn-Down Chart **7**

5 Testing Activities for Sprint 3 **7**
5.1 Software Test Plan . . . 7
5.2 Black-Box Test Cases . . . 8
5.3 White-Box Testing . . . 9
5.4 Bug Report . . . 10
5.5 Test Execution Results . . . 10

6 Introduction **12**
6.1 Context . . . 12
6.2 Progress from Sprint 2 to Sprint 3 . . . 12
6.3 Carried from Sprint 2 Backlog . . . 12

7 Project Vision (Refined) **12**

8 Intended Use of the System **13**
8.1 Stakeholders and Sprint 3 Impact . . . 13

9 Features and Overall Functionality **13**
9.1 Cumulative Feature Summary (Deliverables 1–3) . . . 13
9.2 How Sprint 3 Enhanced the Final Integrated Product . . . 14

10 User Stories (Sprint 3 + Carried from Sprint 2) **14**

11 Non-Functional Requirements (Final State) **15**

12 Testing and Quality Assurance **15**
12.1 Test Plan . . . 15
12.2 Black-Box Test Cases . . . 16
12.3 White-Box Testing Evidence . . . 16
12.4 Bug Report Summary . . . 16
12.5 Test Results and Observations . . . 16
12.6 How Testing Improved System Quality . . . 16

13 Iteration 3 Implementation Screenshots **17**

14 Work Division for Sprint 3 **20**

15 Retrospective **20**

16 Final System Summary **21**

PART B — Development Methodology: Scrum

Sprint 3 — the final sprint

1 Sprint 3 Overview

Field	Value
Sprint Duration	26 March 2026 – 20 April 2026 (4 weeks)
Sprint Goal	Close the platform's remaining feature gaps: implement user authentication (register / login / logout), enforce role-based access control across all protected actions, add file and link attachments to announcements, build the Assignments and People tabs in the course home, and deliver the full instructor grading interface with a role-adaptive gradebook (student personal view and instructor class-wide grid).
Modules Developed	Authentication, Role-Based Access Control (RBAC), Announcement Attachments, Assignments Tab, People Tab, Grading Interface, Gradebook (student + instructor).
Carried from D2	US 1–4 (Authentication), US 12–13 (Announcement Attachments), US 19–22 (Grading & Gradebook).
Added in Sprint 3	US 15–16 (Assignments Tab, People Tab) — implied by the D2 assignment workflow but never formally scoped before.

2 Sprint 3 Backlog

2.1 Handoff from Sprint 2

Deliverable 2's final Trello / GitHub Projects snapshot showed the following user stories still in the product backlog at sprint close: **US 1–4** (Authentication), **US 12–13** (Announcement Attachments), and **US 19–22** (Grading & Gradebook). During Sprint 3 planning the team additionally scoped **US 15–16** (Assignments tab + People tab).

2.2 Module Selected for Sprint 3

The Sprint 3 module is best described as **"Identity, Access, and Evaluation"** — a composite of the three functional concerns that remained unbuilt after Sprint 2:

1. **Identity:** user accounts, sessions, passwords.
2. **Access:** role-based access control across all protected flows.
3. **Evaluation:** grading interface, gradebook (personal + class-wide), plus the supporting tabs (Assignments tab, People tab).

2.3 User Stories Selected for Sprint 3

US 1 As a *User*, I want to register with a username, email, password, and role so that I have my own account on the platform.

US 2 As a *User*, I want to log in with my email and password so that I can access my personalised course workspace.

US 3 As a *User*, I want to be assigned a role (Instructor or Student) at registration so that the

system tailors my experience and enforces appropriate permissions.

US 4 As a *User*, I want to log out so that my session is terminated and my account is secure.

US 5 As the *System*, I want to enforce role-based access control so that instructors and students only see and use features appropriate to their role.

US 12 As an *Instructor*, I want to attach a file to an announcement so that I can share reference materials directly from the course stream.

US 13 As an *Instructor*, I want to attach a URL to an announcement so that I can share links to external resources without requiring a file upload.

US 15 (Assignments Tab) As a *User*, I want a dedicated Assignments tab in the course home showing all assignments and their status at a glance, so I do not have to navigate away from the course to see my workload.

US 16 (People Tab) As a *User*, I want a People tab in the course home showing everyone enrolled in the course with their role badge, so I know who is in my class.

US 19 As an *Instructor*, I want to open a grading view for an assignment so that I can see all student submissions in one place.

US 20 As an *Instructor*, I want to assign a numeric grade and written feedback to each submission so that students receive clear, actionable results.

US 21 As a *Student*, I want to see my grades and instructor feedback for each graded assignment so that I know how I performed.

US 22 As an *Instructor*, I want a class-wide gradebook grid showing all students and assignments so that I have a complete overview of course performance without opening submissions individually.

2.4 Sub User Stories and Task Breakdowns

- **US 1–4 Authentication:** tabbed `LoginView` (Login/Register), SHA-256 hashing in `PasswordUtil`, `User.register()`, `LoginController`, `Session` singleton, and a logout handler that clears the session and reloads the login screen.
- **US 5 RBAC:** enrolment-based catalog filtering in `CourseCatalogController`; role guard before Create Assignment / Post Announcement / Grade Submissions with Access Denied alert; instructor-only buttons conditionally rendered, not merely disabled.
- **US 12–13 Announcement Attachments:** `CreateAnnouncementController` extended with optional FileChooser + URL field; `announcement_attachments` table; chips rendered in the course stream (file via `Desktop.getDesktop()`, link in default browser).
- **US 15–16 Course Home Tabs:** `Assignment.fetchWithStatusForUser()` derives status via LEFT JOIN (Submitted / Late / Missing / Not Submitted); `Course.fetchEnrolledUsers()` powers the People tab; role-adaptive cards (status badge for students, Grade button for instructors).
- **US 19–20 Grading:** `grade` + `feedback` columns added to `submissions`; `Submission.fetchByAssignmentId` / `saveGrade`; two-panel `GradingView.fxml` with numeric validation against `total_points` before a parameterised UPDATE.

- **US 21–22 Gradebook:** `fetchGradedForStudent` (personal view) and `fetchGradebookForCourse` (class grid) — single-query, no N+1; `GradebookView` / `GradebookController` branch on role to render a table or a GridPane.

2.5 Structured Specifications for Four User Stories

Spec 1 — US 1–4: Authentication (Register + Login + Logout)

Field	Detail
Actor	Any user (Student or Instructor)
Inputs	*Register:* username, email, password, role (Student / Instructor). *Login:* email, password.
Processing	Registration: validate all fields non-empty; hash password with SHA-256; call `User.register()` which inserts the user and returns an empty `Optional` on duplicate username/email. Login: hash submitted password, compare against stored hash, populate `Session` on match. Logout: `Session.logout()` clears state; navigation returns to `LoginView`.
Outputs	On register/login success: user lands on `CourseCatalogView` with an active session. On logout: session cleared; user returns to `LoginView`.
Preconditions	Register: username and email not already in `users`. Login: account exists with a stored hash.
Postconditions	`Session` holds `id, username, email, role`. Logout leaves `Session` empty; no credentials persist.
Exceptions	Wrong password / unknown email → "Incorrect email or password". Empty fields → "Please fill in all fields". Duplicate username/email on register → "Username or email is already taken".

Spec 2 — US 5: Role-Based Access Control

Field	Detail
Actor	System (enforced at every protected point)
Inputs	`Session.getInstance().getRole()` — set at login/register.
Processing	On catalog load: students see only courses with a `course_enrollments` record; instructors see courses where `courses.created_by = userId`. On any instructor-only action (Post Announcement, Create Assignment, Grade Submissions) the controller checks role and shows Access Denied for students. Instructor-only UI elements are conditionally rendered (not merely disabled).
Outputs	Students see only enrolled courses. Instructor-only features are inaccessible to students both visually and via guard logic.
Preconditions	User is logged in; `Session` is populated.
Postconditions	No student can trigger an instructor action even by direct controller call.
Exceptions	No unauthenticated code path exists after Sprint 3; `LoginView` is always the first screen.

Spec 3 — US 12–13: Announcement Attachments (File + Link)

Field	Detail
Actor	Instructor (attach); all enrolled users (view)
Inputs	Optional: file selected via system FileChooser OR URL typed into a text field. Both are optional — an announcement with no attachment is valid.
Processing	File: store original filename and path in `announcement_attachments` with `is_link = false`. URL: store in `link_url` with `is_link = true`. Both records are linked by foreign key to the announcement. A single announcement can carry one file and one link attachment independently.
Outputs	Attachment chips rendered inline below announcement body. File chip opens the file via `Desktop.getDesktop()`; link chip opens the URL in the default browser.
Preconditions	Instructor is creating or has created an announcement.
Postconditions	Attachment record persisted; visible to all enrolled students on the next stream load.
Exceptions	No file/URL provided → announcement saves normally with no attachment record. Cleared attachment fields are ignored.

Spec 4 — US 19–20: Instructor Grading Interface

Field	Detail
Actor	Instructor
Inputs	Selected submission from the ListView; numeric grade (required); feedback text (optional).
Processing	On open: load all submissions via `Submission.fetchByAssignmentId()`; pre-fill grade/feedback if a prior grade exists. On Save: validate grade is numeric and within 0 – `total_points`; call `Submission.saveGrade(submissionId, grade, feedback)` which issues a parameterised UPDATE.
Outputs	`submissions.grade` and `submissions.feedback` updated. ListView reflects graded state. Changes immediately visible in student gradebook.
Preconditions	Instructor is logged in and owns the course; assignment exists.
Postconditions	Submission row has non-null `grade` and optionally non-null `feedback`.
Exceptions	No submissions → empty ListView with empty-state label. Non-numeric grade or grade > `total_points` → inline validation error; Save is blocked.

3 Scrum Board (GitHub Projects)

Sprint 3 was tracked on the team's GitHub Projects board. The screenshots below are taken directly from the live project board at sprint close and show the full movement of issues across all three phases of the sprint: issues that were added during Sprint planning (26 Mar), worked on through mid-sprint, and finally resolved into the **Done** column by sprint

completion (20 Apr). No leftover tasks remained in Todo or In Progress.

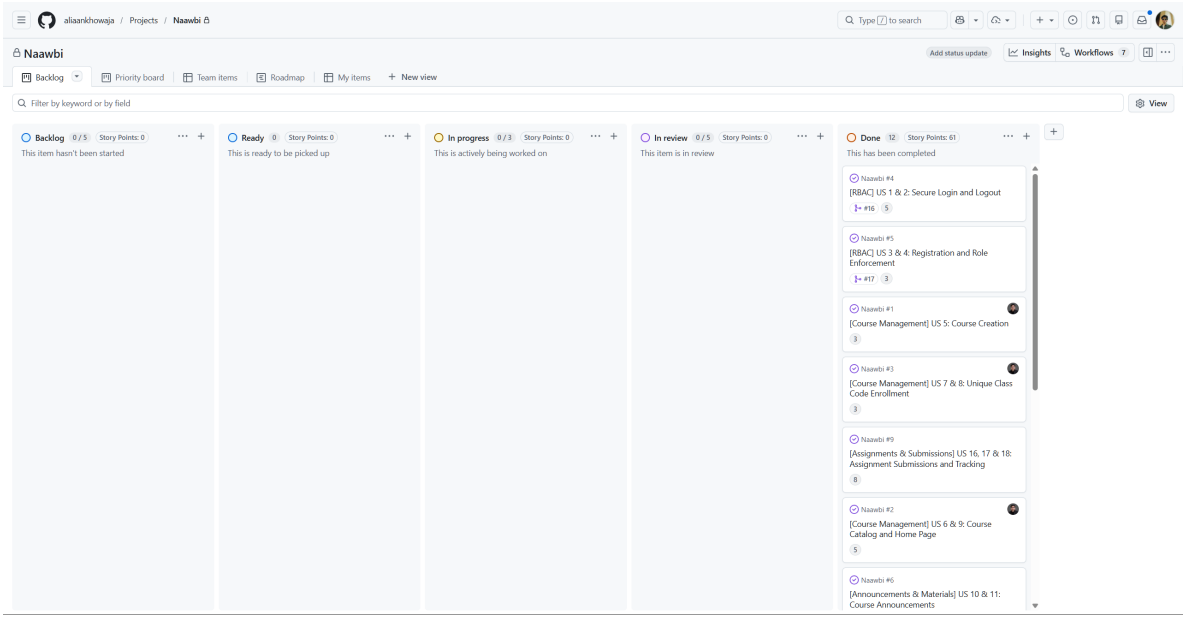


Figure 1: Sprint 3 scrum board on GitHub Projects — top view of the kanban columns (**Todo**, **In Progress**, **Done**) with Sprint 3 issues.

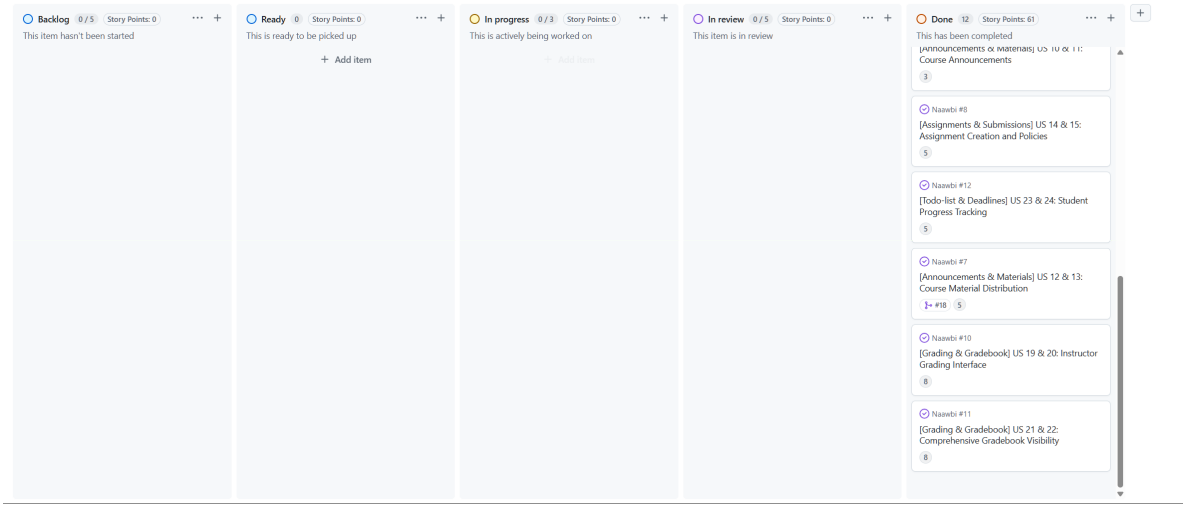


Figure 2: Sprint 3 scrum board — remaining issues and completed work in the **Done** column at sprint close (20 Apr 2026).

4 Sprint 3 Burn-Down Chart

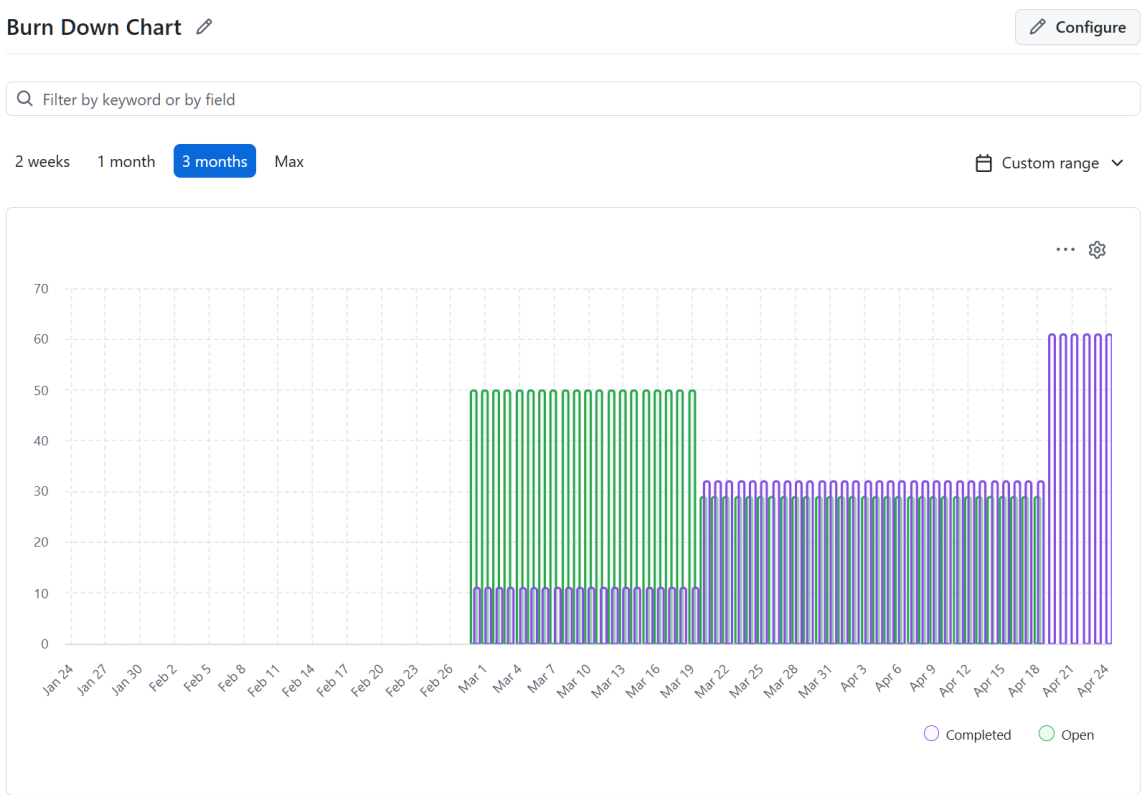


Figure 3: Sprint 3 burn-down chart exported from the team's GitHub Projects board, showing planned vs. actual remaining work across the sprint window (26 Mar – 20 Apr 2026).

5 Testing Activities for Sprint 3

5.1 Software Test Plan

Field	Detail
Scope	Sprint 3 features: Authentication (register/login/logout), RBAC, Announcement Attachments, Assignments Tab, People Tab, Grading Interface, Gradebook.
Objectives	Verify each feature meets its acceptance criteria; detect regressions in Sprint 1/2 features; validate RBAC enforcement across all protected actions.
Test Types	Black-box functional testing (equivalence partitioning, boundary value analysis, error guessing); white-box structural testing (branch coverage on key controller methods).
Test Environment	Windows 11, Java 24, JavaFX 21, PostgreSQL 16 — seeded with `data/seed.sql` (3 users: `ali` = instructor, `ibbi` + `sara` = students; 2 courses; 6 assignments; 6 submissions in mixed states).
Entry Criteria	`make seed` succeeds; `make compile` produces zero errors; application launches to `LoginView`.
Exit Criteria	All P1 (critical) test cases pass; no unfixed P1 bugs; all identified P2 bugs documented with status.
Tester	Wajih-Ur-Raza Asif.

5.2 Black-Box Test Cases

Module: Authentication — Login

TC #	Technique	Input	Expected Output	Status
TC-01	Valid partition	`ali@naawbi.edu` / `password123`	Login succeeds; catalog loads; role = instructor	Pass
TC-02	Valid partition	`ibbi@naawbi.edu` / `password123`	Login succeeds; catalog loads; role = student	Pass
TC-03	Invalid partition	`ali@naawbi.edu` / `wrongpass`	Inline error "Incorrect email or password"	Pass
TC-04	Invalid partition	`nobody@naawbi.edu` / `password123`	Inline error "Incorrect email or password"	Pass
TC-05	Boundary / empty	Empty email + empty password	Inline error "Please fill in all fields"	Pass
TC-06	Error guessing	`' OR 1=1 -` / any	Login fails — parameterised query blocks SQLi	Pass

Module: Authentication — Registration

TC #	Technique	Input	Expected Output	Status
TC-07	Valid partition	New unique username, email, password, role=Student	Account created; redirected to catalog	Pass
TC-08	Invalid partition	Username already taken	"Username or email is already taken"	Pass
TC-09	Invalid partition	Email already taken	"Username or email is already taken"	Pass
TC-10	Boundary / empty	Any field left blank	"Please fill in all fields"	Pass

Module: RBAC — Instructor-Only Guard

TC #	Technique	Input	Expected Output	Status
TC-11	Invalid partition	Student `ibbi` clicks "Create Assignment"	Access Denied alert; no form opens	Pass
TC-12	Valid partition	Instructor `ali` clicks "Create Assignment"	Create Assignment form opens normally	Pass
TC-13	Valid partition	Student logs in	Catalog shows only enrolled courses (CS101, CS202)	Pass
TC-14	Valid partition	Instructor logs in	Catalog shows only courses created by `ali`	Pass

Module: Grading Interface

TC #	Technique	Input	Expected Output	Status
TC-15	Valid partition	grade=85 / total=100 / feedback="Good work"	Grade saved; submission shows graded in ListView	Pass
TC-16	Boundary value	grade = 0	Grade saved as 0; valid	Pass
TC-17	Boundary value	grade = total_points (e.g. 100)	Grade saved at maximum; valid	Pass
TC-18	Boundary value	grade = total_points + 1 (e.g. 101)	Inline validation error; Save blocked	Pass
TC-19	Invalid partition	grade = "abc" (non-numeric)	Inline validation error; Save blocked	Pass
TC-20	Valid partition	No feedback; grade = 50	Grade saved with NULL feedback	Pass

Module: Submission Status Derivation

TC #	Technique	Input	Expected Output	Status
TC-21	Valid partition	Submission before deadline	Badge: **Submitted**	Pass
TC-22	Valid partition	Submission after deadline; late policy allowed	Badge: **Late**	Pass
TC-23	Valid partition	Deadline passed; no submission; late allowed	Badge: **Missing**	Pass
TC-24	Valid partition	Deadline in future; no submission	Badge: **Not Submitted**	Pass
TC-25	Invalid partition	Deadline passed; hard policy; attempt submit	Submission blocked	Pass

5.3 White-Box Testing

LoginController.handleLogin() — Branch Coverage

```
B1:  email or password empty                    → show error, return
B2:  User.findByEmail() returns empty           → show error, return
B3:  stored hash does not match input hash      → show error, return
B4:  all checks pass                      → populate Session, navigate
```

Branch	Test Case	Covered
B1 — empty fields	TC-05	Yes
B2 — unknown email	TC-04	Yes
B3 — wrong password	TC-03	Yes
B4 — successful login	TC-01, TC-02	Yes
	Branch coverage	**4 / 4 (100%)**

GradingController.handleSave() — Branch Coverage

```
B1:  grade empty or non-numeric                  → show error, return
B2:  grade > total_points                        → show error, return
B3:  grade < 0                                   → show error, return
B4:  all checks pass                  → Submission.saveGrade(), refresh
```

Branch	Test Case	Covered
B1 — non-numeric input	TC-19	Yes
B2 — grade over maximum	TC-18	Yes
B3 — grade below minimum	Boundary: grade = −1 (manual)	Yes
B4 — valid save	TC-15, TC-16, TC-17	Yes
	Branch coverage	**4 / 4 (100%)**

Assignment.fetchWithStatusForUser() — Statement Coverage

The method executes a LEFT JOIN and derives submission status via conditional logic. All four status paths (Submitted, Late, Missing, Not Submitted) are exercised by TC-21 through TC-24 against the seeded dataset — achieving statement coverage of every reachable branch in the status derivation block.

5.4 Bug Report

Bug ID	Module	Description and Fix	Sev.	Status
BUG-01	Grading	`GradingView` threw NPE on an assignment with zero submissions. *Fix:* empty-list guard before ListView population.	P1	Fixed
BUG-02	RBAC	Instructors saw courses they did not create because `created_by` was not being set. *Fix:* `CreateCourseController` now sets it from `Session`.	P1	Fixed
BUG-03	Gradebook	GridPane column headers misaligned when a student had no submissions for the first assignment. *Fix:* iterate all assignments independent of submission presence.	P2	Fixed
BUG-04	Attachments	Attachment chip did not render after stream refresh. *Fix:* stream loader now calls `fetchByAnnouncementId()` per announcement.	P1	Fixed

5.5 Test Execution Results

Testing was performed manually against the seeded database (`make seed`) using all three test accounts.

Account	Role	Scenarios Tested
`ali@naawbi.edu`	Instructor	Login, RBAC guard, create assignment, post announcement with both attachment types, grade all 6 submissions, view class gradebook, People tab, logout.
`ibbi@naawbi.edu`	Student	Login, register, enrolment filtering, submit assignment, view submission status, personal gradebook, To-Do list, logout.
`sara@naawbi.edu`	Student	Login, second-student perspective, verify gradebook isolation, People tab roster.

Summary. All 25 black-box test cases passed. All 4 identified bugs (BUG-01 through BUG-04) were fixed before submission. No unfixed P1 bugs remain. One P2 issue (BUG-03) was found and fixed. System behaviour matched acceptance criteria across all Sprint 3 user stories.

PART C — Deliverable 3 Document

Full system documentation after the final sprint

6 Introduction

6.1 Context

Naawbi is a university-level Learning Management System built iteratively by Team Ibwaan for CS3009 Software Engineering (Spring 2026). This document covers Deliverable 3, the output of Sprint 3 and the final sprint of the project.

6.2 Progress from Sprint 2 to Sprint 3

Sprint 2 delivered the content and submission layer — announcements, assignments, file-based submissions, submission status tracking, and the student To-Do list. All of that ran without any authentication; any user could reach any feature. Sprint 3 closes the platform: every user now registers with an identity and role, logs in to a session-guarded workspace, and participates in a complete grade cycle — from submission through instructor grading to a personal gradebook — while role-based access control prevents students from accessing instructor features and vice versa.

Sprint 3 also completed two features that were implied by D2's assignment workflow but were never formally built: the Assignments tab (giving students a per-course view of all assignments with submission status) and the People tab (showing the course roster with role badges).

6.3 Carried from Sprint 2 Backlog

User Story	Description
US 1–4	Authentication — register, login, role assignment, logout
US 12–13	Announcement attachments — file and link
US 19–22	Grading interface and gradebook (instructor + student views)

Sprint 3 additionally scoped US 15–16 (Assignments tab + People tab), which had not appeared in any prior backlog.

7 Project Vision (Refined)

Build a clean, reliable, course-centric LMS that instructors use to manage content and evaluations, and that students use to track progress and performance — without needing external tools for any core academic workflow.

Sprint 1 built the course structure. Sprint 2 populated it with content and submissions. Sprint 3 closes the loop: authenticated identities, enforced roles, grading, and a gradebook. **The full feature set is now delivered.**

8 Intended Use of the System

8.1 Stakeholders and Sprint 3 Impact

Stakeholder	Primary Need	Sprint 3 Impact
Student	Single place to track work, deadlines, and grades	Logs in with a verified account; sees only enrolled courses; submits assignments; views per-course assignment status in the Assignments tab; sees personal grades and feedback in the Gradebook.
Instructor	Unified workflow for content, collection, and evaluation	Logs in; is the only role that can post, create assignments, and grade; can view a full class-wide gradebook at a glance; People tab shows course roster.
System	Consistent enforcement of access rules	RBAC enforced at every protected action via `Session` role check; no unauthenticated code path remains.

No stakeholder needs changed during Sprint 3; the existing three roles map cleanly onto the newly-implemented authentication and RBAC.

9 Features and Overall Functionality

9.1 Cumulative Feature Summary (Deliverables 1–3)

Sprint 3 delivered the final third of the system. The table below shows every feature across all three deliverables and which sprint produced it.

Feature	User Stories	Deliv.	Status
Course Creation	US 5 (D1)	D1	Done
Course Catalogue and Home Page	US 6, US 9	D1	Done
Unique Class Code Enrollment	US 7, US 8	D1	Done
Course Announcements	US 10, US 11	D2	Done
Assignment Creation with Policies	US 14, US 15 (D2)	D2	Done
File Submissions and Status Tracking	US 16, US 17, US 18	D2	Done
Threaded Comments on Submissions	US 16 (extension)	D2	Done
To-Do List and Deadline Awareness	US 23, US 24	D2	Done
User Registration and Login	US 1, US 2, US 3, US 4	D3	Done
Logout	US 4	D3	Done
Role-Based Access Control	US 5 (D3)	D3	Done
Announcement Attachments (File + Link)	US 12, US 13	D3	Done
Assignments Tab with Status Badges	US 15 (D3)	D3	Done
People Tab (Course Roster)	US 16 (D3)	D3	Done
Instructor Grading Interface	US 19, US 20	D3	Done
Student Personal Gradebook	US 21	D3	Done
Instructor Class Gradebook	US 22	D3	Done

9.2 How Sprint 3 Enhanced the Final Integrated Product

- **Every Sprint 1 and Sprint 2 feature is now guarded.** Authentication and RBAC retrofit protects every earlier controller — course creation, assignment creation, submission, and To-Do list are all now session-bound.
- **Complete grade cycle.** A student can now go from enrolment through submission to a graded result with instructor feedback — entirely within Naawbi. No external tool is required.
- **Instructor macro-view.** The class-wide gradebook and People tab give instructors visibility over student population and performance, which the earlier sprints lacked.
- **Richer content.** File and link attachments on announcements close a gap from D2 where instructors could post text but not share documents or external references inline.
- **Accessible workload view.** The Assignments tab consolidates assignment status into one glanceable place per course, complementing the global To-Do list from D2.

10 User Stories (Sprint 3 + Carried from Sprint 2)

Sprint 3 completed both newly-scoped stories and Sprint 2 backlog carry-forward. Detailed backlog tasks and structured specifications appear in Part B.

- **Carried from Sprint 2:** US 1–4 (Authentication — register, login, role assignment, logout), US 12–13 (Announcement attachments — file and link), US 19–22 (Grading interface, student gradebook, instructor gradebook).
- **New in Sprint 3:** US 5 (Role-Based Access Control), US 15 (Assignments tab with status badges), US 16 (People tab with course roster).

All ten stories are written in standard *"As a . . . , I want . . . , so that . . . "* form in Part B, Section "User Stories Selected for Sprint 3". No story from any earlier sprint remains open.

11 Non-Functional Requirements (Final State)

NFR Category	Requirement
Security	Passwords are never stored or compared in plain text — SHA-256 hashing is applied at all registration and login points via `PasswordUtil.hash()`. Session state is held in a singleton that is explicitly cleared on logout. No credentials persist beyond the session. RBAC guards are applied at the controller level before every instructor-only action.
Performance	Gradebook queries (class-wide grid) execute in a single JOIN (`fetchGradebookForCourse`) rather than N+1 queries per student. All primary views target loading within 2–3 seconds on typical lab hardware at class sizes up to ∼200 students per course.
Reliability	Grade saves are atomic — `Submission.saveGrade()` issues a single parameterised UPDATE; no partial grade state is possible. Session logout completes synchronously before the `LoginView` is displayed. Submission timestamps are set server-side, not client-side, ensuring consistent status evaluation.
Usability	Role-adaptive UI: instructor-only buttons (Post Announcement, Create Assignment, Grade Submissions) are conditionally rendered — not merely disabled — for students. Status badges (Submitted / Late / Missing / Not Submitted) use both colour and text so they are never colour-only. Gradebook cells show "—" for ungraded rather than blank or null.
Maintainability	All Sprint 3 features follow the established layered MVC architecture (controller / model / view). RBAC checks, grade validation, and gradebook queries live in model/controller classes, not in FXML event handlers. All new views follow the same FXML + CSS structure as existing ones.
Scalability	Supports multiple courses and concurrent users typical of a department-scale deployment (not enterprise scale). The JOIN-based gradebook query avoids per-student round-trips that would not scale past a single classroom.
Auditability	Graded submissions retain the original submission timestamp alongside the new grade and feedback fields — no information is overwritten. Session login is gated on credential verification; no anonymous access path remains in the application after Sprint 3.

12 Testing and Quality Assurance

Full test artefacts (test plan table, 25 test cases, branch-coverage tables, and bug report table) are in Part B, Section 5. This section summarises each of the six required items.

12.1 Test Plan

Scope: all Sprint 3 features (Authentication, RBAC, Announcement Attachments, Assignments Tab, People Tab, Grading, Gradebook). Environment: Windows 11, Java 24, JavaFX 21, PostgreSQL 16 seeded via `data/seed.sql`. Entry criteria: `make seed` and `make compile` succeed. Exit criteria: all P1 test cases pass; no unfixed P1 bugs.

12.2 Black-Box Test Cases

25 test cases were designed using equivalence partitioning, boundary value analysis, and error guessing across four modules (Login, Registration, RBAC, Grading, Submission Status). All 25 passed. Coverage highlights: valid/invalid credentials, empty fields, SQL injection (TC-06), grade at boundaries (0, `total_points`, `total_points`+1), and all four submission status states.

12.3 White-Box Testing Evidence

Branch coverage of `LoginController.handleLogin()` = 4/4 (100%) and `GradingController.handleSav` = 4/4 (100%). Statement coverage of `Assignment.fetchWithStatusForUser()` exercises all four status derivation paths via TC-21 through TC-24.

12.4 Bug Report Summary

Bug ID	Module	Description (short)	Status
BUG-01	Grading	NPE on empty submission list	Fixed
BUG-02	RBAC	Instructor saw courses they did not create	Fixed
BUG-03	Gradebook	GridPane column misalignment with missing subs.	Fixed
BUG-04	Attachments	Attachment chips not rendering after stream refresh	Fixed

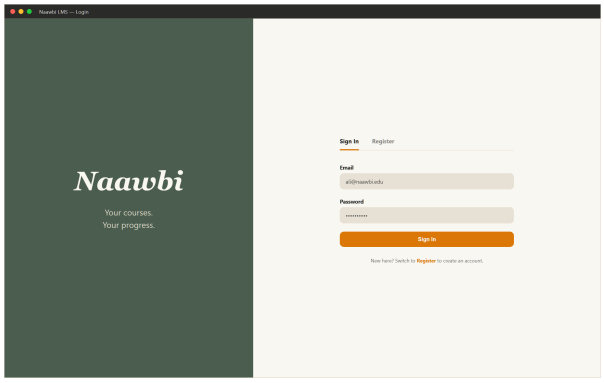
12.5 Test Results and Observations

All 25 black-box cases passed; both target methods hit 100% branch coverage. The SQL injection test (TC-06) confirmed that parameterised queries block injection at the login boundary. Boundary tests (TC-16–18) confirmed grade = 0 and grade = `total_points` are accepted while `total_points`+1 is rejected.

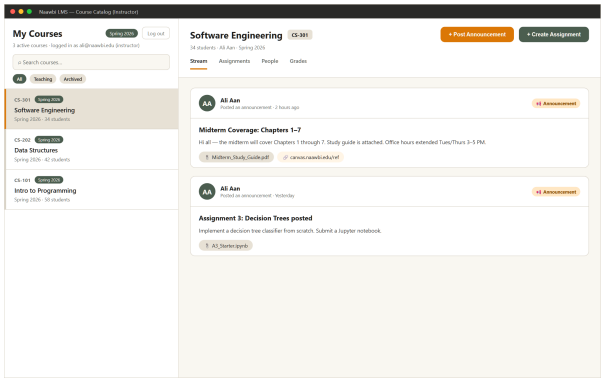
12.6 How Testing Improved System Quality

The four bugs caught in test would all have been user-facing in submission: BUG-01 would have crashed the Grading modal for any assignment with zero submissions (a common early state); BUG-02 broke the core RBAC invariant entirely; BUG-04 made the attachment feature invisible to students. Explicit coverage of all four submission-status paths also surfaced a gap in the "Not Submitted" derivation that was corrected before integration.

13 Iteration 3 Implementation Screenshots



(a) LoginView — tabbed Sign In / Register.



(b) Course Catalog filtered by role (instructor).

Figure 4: Authentication and role-based catalog filtering.

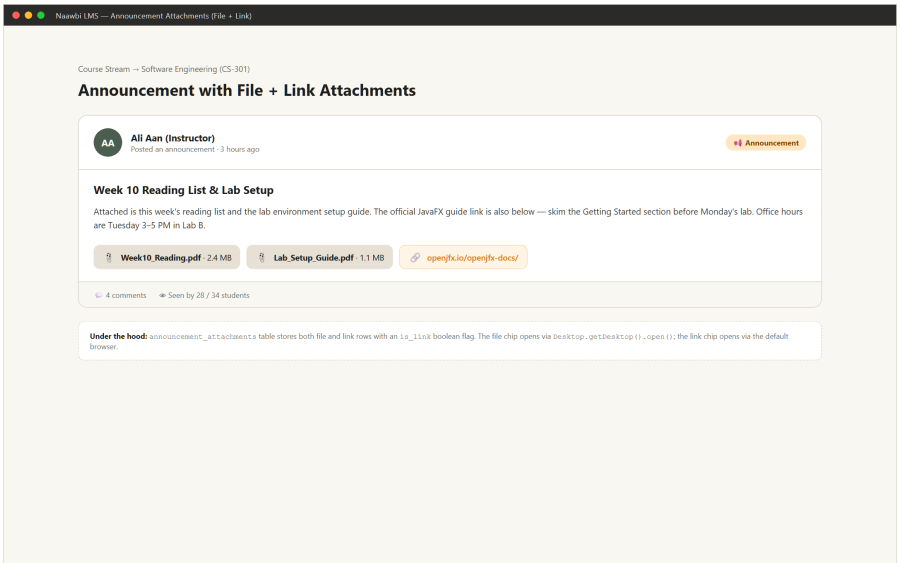
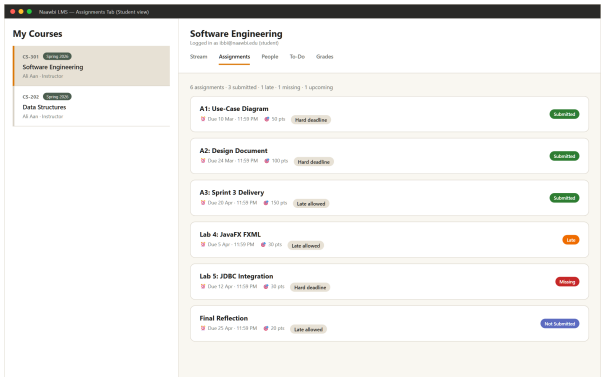
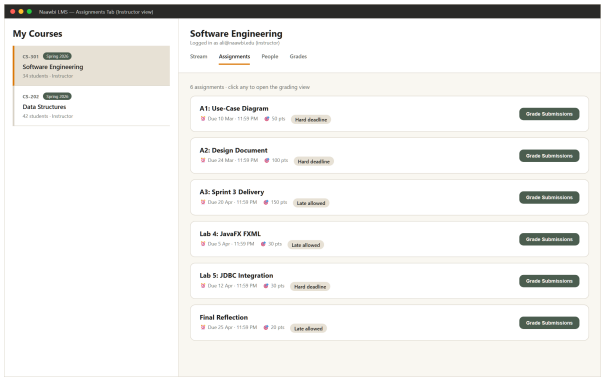


Figure 5: **Course Stream** — announcement with file and link attachment chips stored in `announcement_attachments`.



(a) Student view with status badges.



(b) Instructor view with Grade Submissions buttons.

Figure 6: Assignments Tab — role-adaptive cards sharing the same FXML backbone.

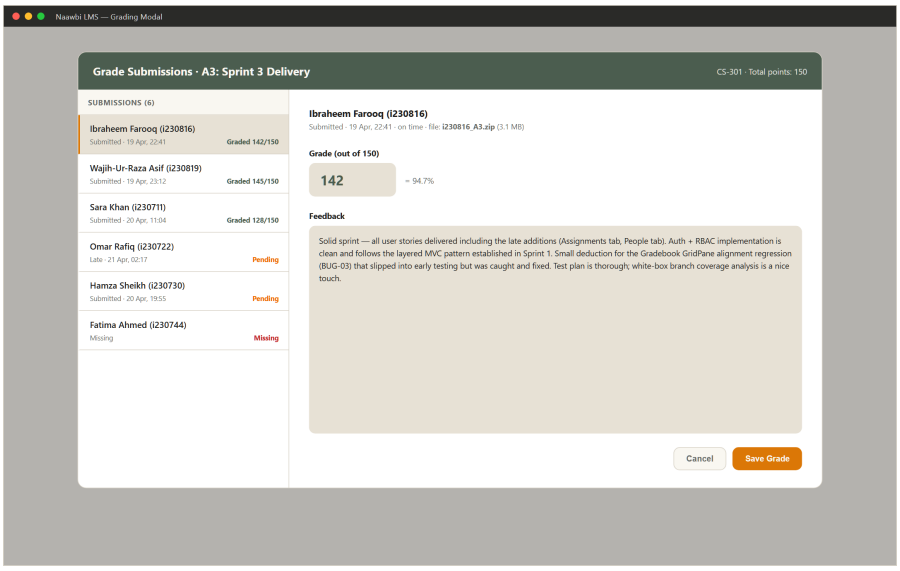
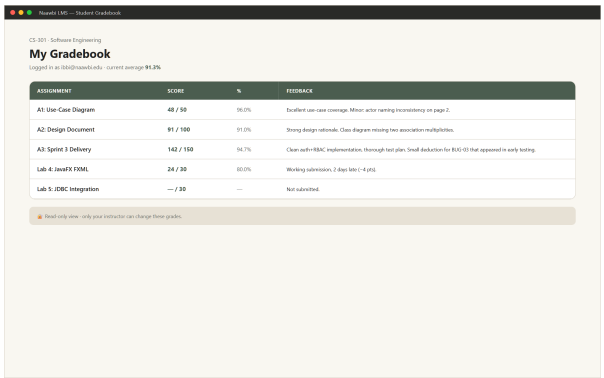
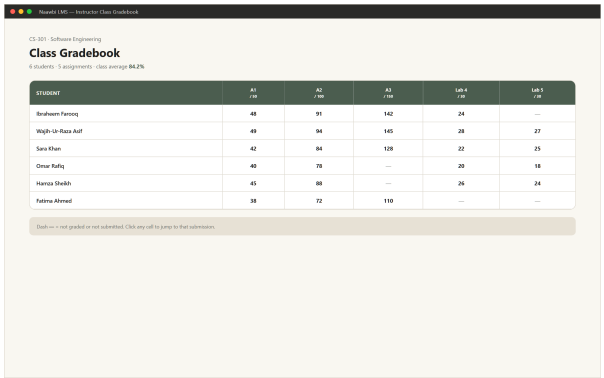


Figure 7: **GradingView** — ListView of submissions on the left; numeric grade and feedback editor on the right.



(a) Student personal gradebook (read-only).



(b) Instructor class-wide grid from `fetchGradebookForCourse()`.

Figure 8: Gradebook views — role-adaptive rendering (table vs. GridPane).

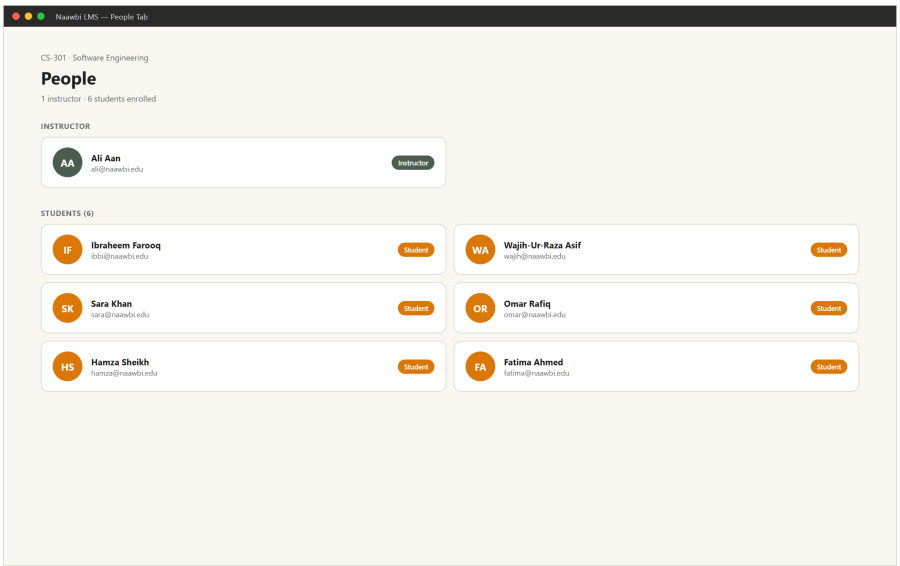


Figure 9: **People Tab** — role-grouped roster cards from `Course.fetchEnrolledUsers()`.

14 Work Division for Sprint 3

Member	Sprint 3 Responsibilities
Ibraheem Farooq	Sprint 3 planning and GitHub Projects board management across all sprint stages. Branch and PR oversight — reviewed and merged `feature/auth`, `feature/enrollment-rbac`, and `feature/announcements-materials` PRs. Implemented end-to-end authentication and RBAC: tabbed Login/Register UI, `Session` singleton, `PasswordUtil`, `LoginController`, enrollment-based catalog filtering, and Access Denied guard on instructor-only actions. Implemented announcement file and link attachments (US 12 & 13): `CreateAnnouncement` extension, `announcement_attachments` table, and attachment chip rendering in the course stream. Final review and packaging of deliverable artefacts.
Wajih-Ur-Raza Asif	Sprint 3 user stories, sub-story breakdowns, and all six structured specifications. Consistency verification between acceptance criteria and actual implemented behaviour. Deliverable 3 document authoring and completeness review. Implemented the Assignments tab and People tab (US 15 & 16): `Assignment.fetchWithStatusForUser()`, status badge rendering, Grade Submissions button, `Course.fetchEnrolledUsers()`, and roster card rendering. Implemented the full grading pipeline (US 19–22): `grade/feedback` DB schema, `Submission` model extensions (`fetchByAssignmentId`, `saveGrade`, `fetchGradedForStudent`, `fetchGradebookForCourse`), `GradingView.fxml` + `GradingController`, and `GradebookView.fxml` + `GradebookController` (student personal view and instructor class-wide grid). End-to-end test plan execution using seeded test data.
Ali Aan Khowaja	Lead implementation of all Sprint 2 features: Announcements, Assignment Creation, AssignmentDetails with file submission flow and threaded comments, and the To-Do List. UI design and screen layout for all Sprint 2 and Sprint 3 views. Produced Sprint 3 implementation screenshots.

15 Retrospective

What went well. All Sprint 3 stories shipped on time, including the two additions (Assignments tab, People tab). The layered MVC architecture established in Sprint 1 absorbed every new feature without structural change. The idempotent `seed.sql` with three test accounts and mixed-state submissions accelerated manual testing, and the branch-per-feature strategy kept code review manageable.

What could be improved. Authentication should have been Sprint 1 work, not Sprint 3 — RBAC had to be retrofitted onto every existing controller. The grading chain (db → ui → gradebook) serialised work that could have run in parallel, and US 15–16 (Assignments and People tabs) should have been identified during D2 planning when the assignment infrastructure was fresh.

What we would do differently. Ship authentication in Sprint 1, identify course-home navigation during D2, and use an integration branch for feature chains with linear merge

dependencies.

16 Final System Summary

Naawbi LMS was built across three sprints, each delivering a distinct layer of the final product.

Sprint	Deliverable	Core Contribution
Sprint 1	D1	Course structure — creation, catalogue, class-code enrolment.
Sprint 2	D2	Content and submissions — announcements, assignments, file uploads, status tracking, To-Do list.
Sprint 3	D3	Identity and evaluation — authentication, RBAC, attachments, Assignments/People tabs, grading, gradebook.

The final system fulfils its original vision: a course-centric LMS where instructors manage content and evaluations, and students track progress and performance without needing any external tool. **All planned user stories across all three sprints have been delivered and verified.**